

Slash Commands Deep-Dive & CI/CD Workflows

1. Command anatomy — what goes inside a command file

A well-designed slash command file has four sections. The exam tests whether you know what belongs in each.

Example: `.claude/commands/deploy-check.md`

```
# /deploy-check

## Purpose

Verify the deployment is safe to proceed before merging to main.

## Context

- Project uses blue-green deployment strategy
- Production DB migrations must be backward-compatible
- Rollback window is 15 minutes

## Steps

1. Check all tests pass in CI
2. Verify migration files are backward-compatible
3. Confirm no breaking API changes vs current production
4. Check deployment size – alert if > 200MB
5. Output a go/no-go decision with reasoning

## Output format

JSON with keys: decision, reasons[], warnings[], rollback_risk
```

Good command design

- Single clear purpose — one command does one thing.
- Explicit output format — specify JSON structure.
- References project context from CLAUDE.md.
- Numbered steps for consistency across runs.
- Includes edge cases and pass/fail criteria.

Bad command design — avoid these

- Vague instructions like 'check everything' with no criteria.
- No output format specified — produces unparseable natural language.
- Hardcoded file paths that won't work on other machines.
- Too many responsibilities — split into multiple commands.
- No criteria for what constitutes pass vs fail.

2. Arguments & context passing

Slash commands can accept arguments — the text after the command name is passed as natural language context to Claude.

```
# Usage examples

/review src/auth/login.ts # filename as argument
/security-check payments/ # directory as argument

/migrate --dry-run # flag as argument

/test unit # scope as argument
```

How arguments work — exam-critical rule

- Arguments are passed as natural language context — NOT formal parameters.
- There is no \$1, {arg}, or %s syntax in slash command files.
- The command file describes in plain English how to use the argument.
- Example: 'Review the file or directory provided as the argument.'
- If no argument is provided, the command file should define a sensible default.

3. Chaining commands

Commands can be designed to chain — one command's output feeds the next. This is the slash command equivalent of a sequential pipeline. Each command should produce structured output (ideally JSON) that the next command can consume as input context.

#	Command	Purpose	Output	Feeds into
1	/lint	Check code style	{pass: bool, issues: []}	/test
2	/test	Run test suite	{pass: bool, failures: []}	/security-check
3	/security-check	Audit for vulnerabilities	{pass: bool, severity: str}	/deploy-check
4	/deploy-check	Final go/no-go decision	{decision: str, risk: str}	Pipeline gate

4. Exit codes — the CI contract

Exit codes are how Claude Code communicates pass/fail to the CI pipeline. This is a contract — the pipeline trusts the exit code, not the text output. Claude does NOT automatically exit with non-zero on failure — you must explicitly instruct it.

Exit code	Meaning	Pipeline action
0	Success — task completed, no blocking issues	Continue to next step
1	Failure — blocking issues found	Fail the build, block merge
Non-zero	Any failure — same as exit 1 in most pipelines	Fail the build

5. The three CI/CD design principles

Always request structured output

- Natural language output is unreadable by pipelines — always ask for JSON.
- Specify the exact JSON structure: {pass: bool, issues: [], severity: string}.
- Pipelines read exit codes and structured data — never prose.
- Example: "Output your findings as JSON: {safe: bool, issues: [], blockers: []}"

Pre-authorise irreversible actions explicitly

- In CI, there is no human to confirm — Claude cannot pause and ask.
- If the pipeline needs Claude to deploy, delete, or modify — pre-authorise in the prompt.
- Example: "You are authorised to apply this migration to staging. Do not ask for confirmation."
- This is the CI equivalent of the minimal footprint principle — bake authorisation in.

Make prompts environment-aware

- CI runs in a different environment than local dev.
- Reference environment variables, not hardcoded paths.
- Example: `claude --print "Review $CHANGED_FILES against standards in CLAUDE.md"`
- API keys must always come from secrets — never hardcode in workflow files.

6. GitHub Actions integration — exam pattern

The correct GitHub Actions pattern for Claude Code CI integration:

```
# .github/workflows/claude-review.yml
name: Claude PR Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Claude Security Review
        run: |
          claude --print "Review changed files for security vulnerabilities.
          Output JSON: {pass: bool, issues: [], severity: string}
          Exit with code 1 if high severity issues found." > review.json
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

What the exam tests about this pattern

- --print flag for headless operation — not --ci, --interactive, or --headless.
- API key from secrets — `{{ secrets.ANTHROPIC_API_KEY }}` — never hardcoded.
- Exit code gates the pipeline — explicitly instructed in the prompt.
- JSON output — redirected to review.json for artifact storage.
- CLAUDE.md project standards apply automatically — no special config needed.

7. The CI/CD prompt checklist — use on every exam question

When reviewing any CI prompt scenario in the exam, check all four. If any fail — that option is wrong.

Check	Question to ask	Correct answer
Flag	Is the correct CI flag used?	--print (not --ci, --interactive, --headless)
Exit code	Does the prompt instruct Claude to exit non-zero?	Must be explicitly stated — Claude does not auto-fail
Output format	Is structured output requested?	JSON — pipelines cannot parse natural language
API key	Where does the API key come from?	Secrets/env vars — never hardcoded in workflow file

8. Exam-critical summary table

Concept	Key rule
Command arguments	Passed as natural language context — no <code>\$1</code> , <code>{arg}</code> , or formal syntax
Command chaining	Sequential pipeline — structured JSON output from one feeds the next
CI output format	Always JSON — pipelines cannot parse natural language
Exit codes	Must be explicitly instructed — Claude does not auto-fail
Pre-authorisation	Bake confirmation into the CI prompt — no human available in CI
API key in CI	Always from secrets/env vars — never hardcoded in workflow files
Good command design	Single purpose, explicit output format, numbered steps, pass/fail criteria

Bad command design

Vague instructions, no output format, hardcoded paths, multiple responsibilities

9. Quick recall — 3 things to remember

1 Always JSON in CI.

Pipelines read exit codes and structured output — never natural language. Every CI prompt should specify a JSON output format.

2 Exit codes must be explicitly instructed.

Claude does not automatically exit 1 on failure. You must tell it: 'Exit with code 1 if issues are found.'

3 Pre-authorise irreversible actions.

No human confirmation is possible in CI. Bake the authorisation into the prompt itself — do not rely on Claude to pause and ask.

10. Practice questions & answers

Three questions from today's session. Q2 was answered partially — always check for multiple problems in CI prompt scenarios.

Q1. A team created a `/deploy-check` slash command. A developer runs: `/deploy-check src/payments/checkout.ts`. How does Claude Code interpret the text '`src/payments/checkout.ts`' that follows the command name?

- A) It is ignored — slash commands do not support arguments
- B) It is passed as a formal positional parameter mapped to `$1` inside the command file
- ✓ C) It is passed as natural language context that Claude uses when executing the command instructions
- D) It triggers a file upload — Claude Code reads the file content automatically

Explanation:

- C is correct: Arguments are treated as natural language context — the text after the command name is simply appended to the conversation context. There is no formal parameter syntax like `$1` or `{arg}`. The command file describes in plain English how to use the argument.
- A is wrong: Slash commands absolutely support arguments.
- B is wrong: No formal positional parameter syntax exists in slash command files.
- D is wrong: The argument is text context, not a file upload trigger.

Q2. A DevOps engineer writes this CI step: `claude --print 'Check if the deployment is safe to proceed. Let me know what you find.'` After running, the CI pipeline always continues even when Claude finds critical issues. What is wrong with this CI prompt design?

Your answer: B

- A) The `--print` flag is incorrect — should use `--ci` for pipeline integration
- B) The prompt does not instruct Claude to exit with a non-zero code when critical issues are found
- C) The output format is unstructured — pipelines cannot parse natural language responses
- ✓ **D) Both B and C — the prompt is missing both exit code instructions and a structured output format**

Explanation:

- D is correct: There are TWO problems. Problem 1 (B): Claude does not auto-exit with non-zero on failure — must explicitly instruct 'Exit with code 1 if critical issues found'. Problem 2 (C): 'Let me know what you find' produces natural language — pipelines cannot parse prose. Must specify JSON output format.
- B alone is wrong: Correct diagnosis but misses the equally important output format problem.
- C alone is wrong: Correct diagnosis but misses the exit code problem.
- A is wrong: `--print` is correct. There is no `--ci` flag.
- Exam strategy: When multiple problems exist, always check if a combined answer option is available before selecting a single-issue option.

Q3. Which GitHub Actions step correctly implements a Claude Code PR security review that blocks merges on high severity issues and stores output as an artifact?

- A) `claude --interactive 'Review for security issues'`
- ✓ **B) `claude --print '...Output JSON...Exit with code 1 if high severity...' > review.json`
`env: ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }`**
- C) `claude --print '...Output JSON...Exit with code 1...' > review.json` `env: ANTHROPIC_API_KEY: sk-ant-hardcoded-key`
- D) `claude --ci 'Review for security vulnerabilities' exit 0`

Explanation:

- B is correct: Uses `--print` (headless), requests JSON output, instructs exit 1 on high severity issues, redirects to `review.json` for artifact storage, loads API key from GitHub secrets. All four CI checklist items pass.
- A is wrong: `--interactive` does not exist. Interactive mode requires a terminal — CI pipelines have no terminal.
- C is wrong: Hardcoding the API key is a critical security violation. Anyone with repo access can see it. Always use `${ secrets.ANTHROPIC_API_KEY }`.
- D is wrong: `--ci` flag does not exist. `exit 0` at the end overrides Claude's exit code — the pipeline can never fail even with critical issues found.

Day 7 scorecard

2 / 3

Good session — check for multiple problems in CI scenarios

Slash command arguments — natural language context	✓ Correct
CI prompt design — exit codes + output format	✗ Review
GitHub Actions integration — full correct pattern	✓ Correct

Key takeaway from missed Q2 — multiple problems in one scenario

- When a CI prompt scenario has issues, always check ALL four items on the checklist.
- Flag correct? Exit code instructed? JSON output requested? API key from secrets?
- The exam regularly presents TWO problems in one scenario with a combined answer option (e.g. 'Both B and C').
- Strategy: spot one problem → immediately ask 'what else is wrong?' before selecting your answer.
- Selecting a partial answer (B only) when D (both B and C) exists is a common exam mistake.

Next up: Day 8

MCP fundamentals — what is the Model Context Protocol, how MCP servers work, and tool boundary design.